OAP200V

Object-oriented analysis, design and programming

00 – Eclipse environment

Lecturer: Boban Vesin

Outline

- Java environment
- Eclipse IDE tutorial
  - Basic features
  - Installation
  - Quick start guide

The Eclipse IDE

- IDE = “Integrated Development Environment”
- Contains all the utilities needed to write, build, test, and publish source code
- Has deep integration with specific programming languages
  - Can detect errors before they occur
  - Helps maintain good documentation for your code, and provide easy-to-access references
  - GUI builder
  - Version control
  - etc

ECLIPSE IDE - Introduction

- One of the most used IDEs available for programming in different languages.
  - E.g. : JAVA, HTML, PYTHON, PERL, COBOL etc.
- Eclipse is open-source software and is freely available online.
  - https://www.eclipse.org/downloads/
- Eclipse Foundation is developing and producing different versions of Eclipse IDE.
- The most recent version is Eclipse IDE 2026-06 R.
  - Older versions released are Kepler, Luna, Mars, Neon, Oxygen, Photon etc.
  - From 2018 releases are named **YYYY-MM**
- Eclipse Public License.

More…

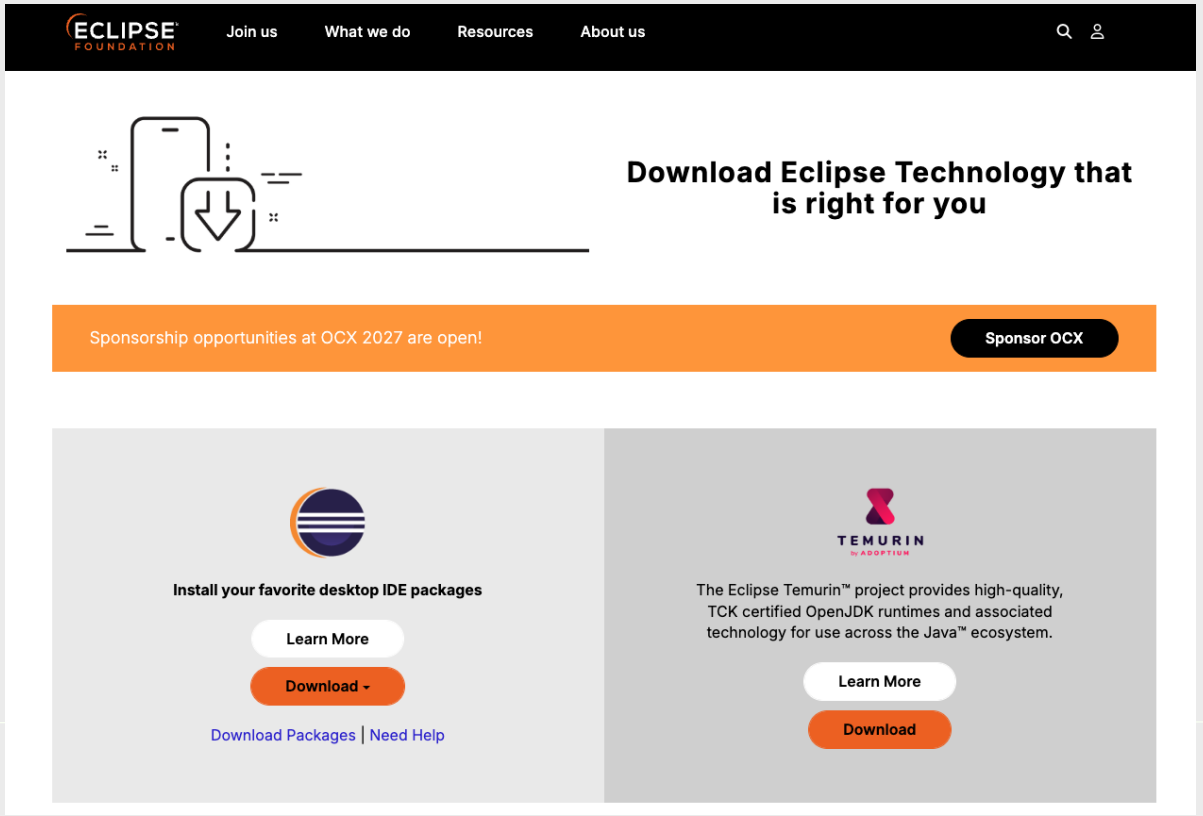
- Eclipse supports coding in multiple languages in single package.
  - E.g. : JavaScript in HTML, CSS in HTML etc.
- Runs on Windows, Linux, Max OS and Solaris.
- Supports extensions and third-party plug-ins after installation.
  - E.g. : Android application development with the help of ADT plug-in.
- Supports other functionalities for clean code.
  - E.g. : Auto formatter, Syntax checking, Spell check, Debugging etc.
- Supports Concurrent version systems that are provided by third party plug-ins.
  - E.g. : GitHub Mylyn Connector
- Supports program debugging tools.

Installation

Downloading Eclipse

- Go to https://www.eclipse.org/downloads/

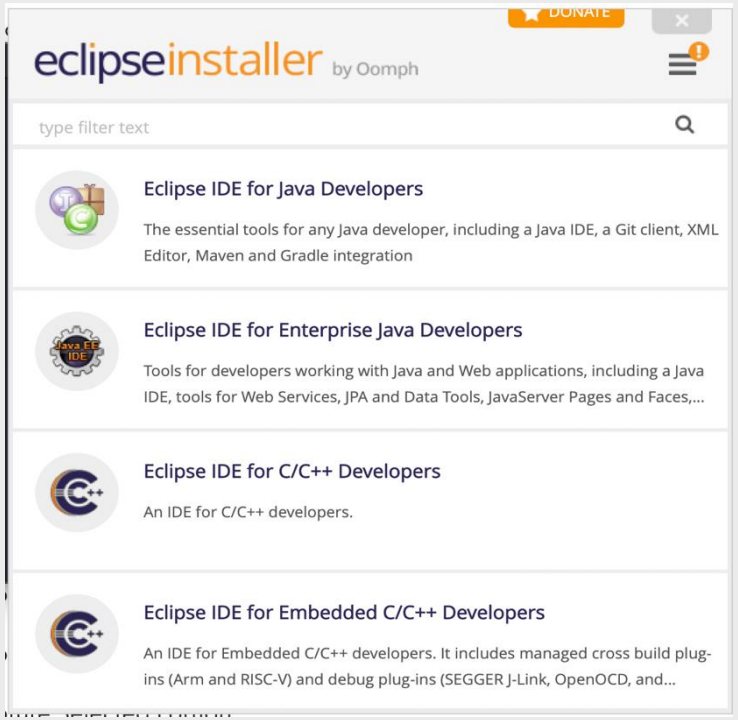


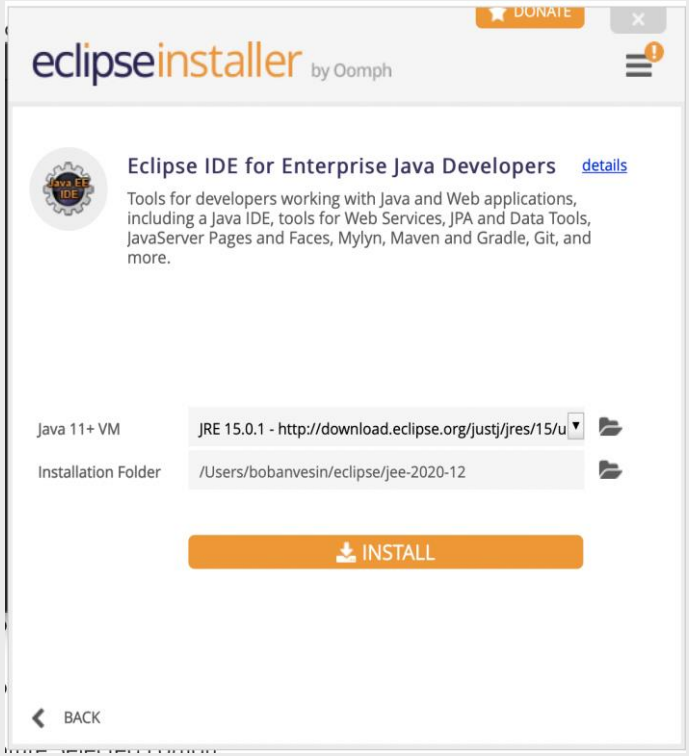


Install Eclipse (Mac, Windows)

- Run Eclipse Installer.

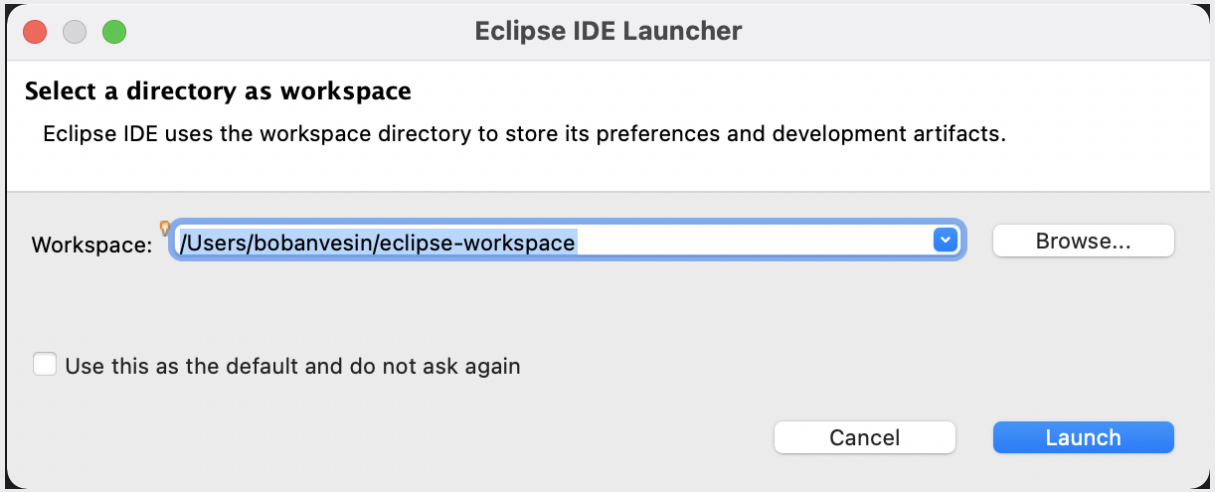






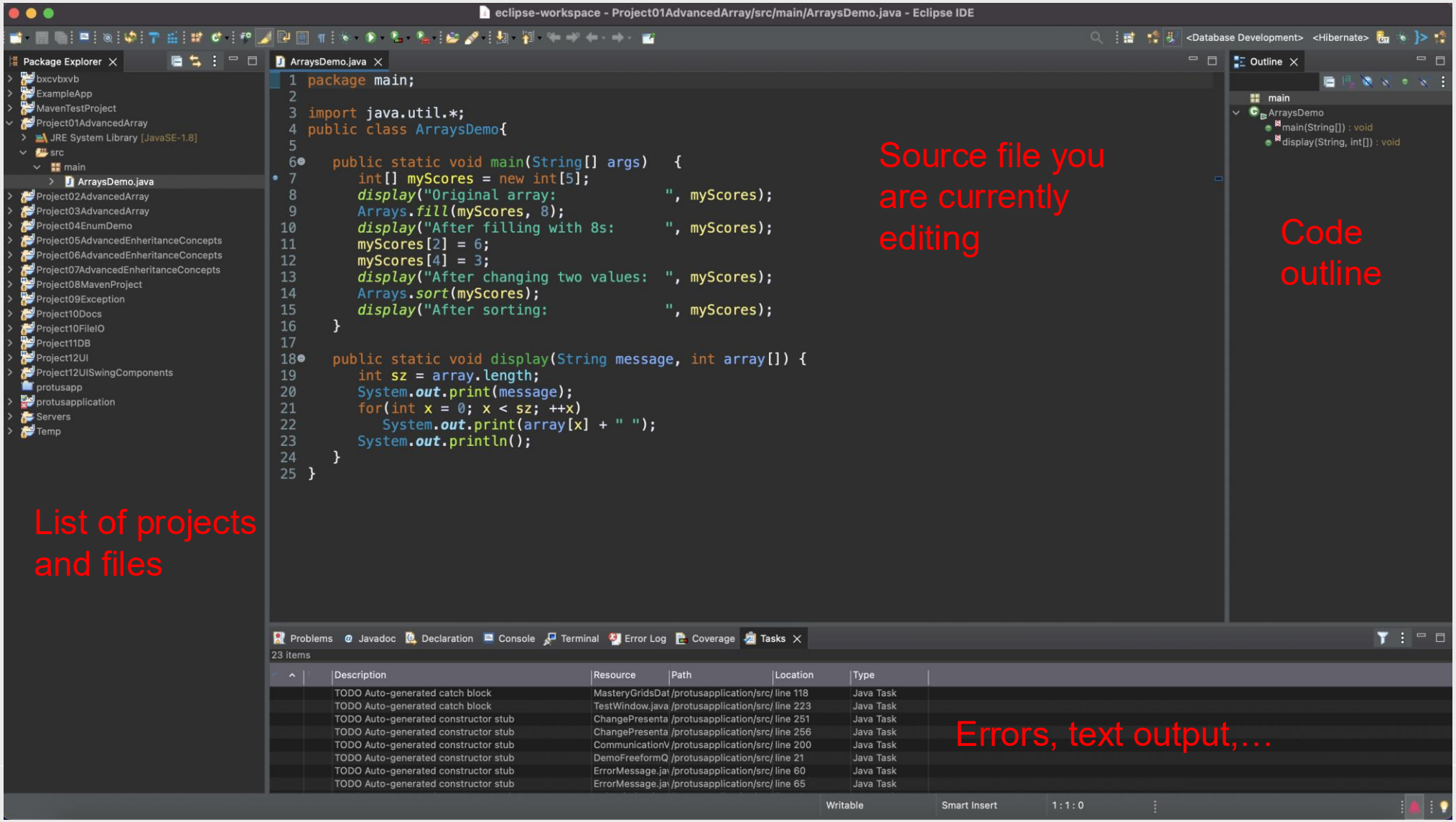
Run Eclipse (Mac, Windows)

- Open Eclipse and select a folder to store your projects in Workspace Launcher



Features

Eclipse IDE (Mac)



Syntax checking, Spell checking

*FavouriteToy.java

```
public class FavouriteToy {

   // Since I have only one class,
   // I must include the "public static main()" method in this class.
   public static void main(String[] args) {

      // Make an object to hang my "printToy()" method on.
      Syntax error, insert ";" to complete BlockStatements eToy()

      // Check the number of arguments received as input.
      // Default to "purple koalas" if number is not 2.
      if (args.length == 2) {
         favToy.printToy(args[0], args[1]);
      } else {
         favToy.printToy("purple", "koalas");
      }


   // Construct and print a text string showing the favourite colour and toy
   private void printToy(String colour, String toy) {
      String s = String.format("My favourite toys are %s %s.", colour, toy);
      System.out.println(s);
   }
}
```

ProcessorExpert.c

```
86
87 /*lint -save  -e970 Disable MISRA rule (6.3) checking. */
88 int main(void)
89 /*lint -restore Enable MISRA rule (6.3) checking. */
90 {
91   /* Write your local variable definition here */
92
93   /*** Processor Expert internal initialization. DON'T REMOVE THIS CODE!!! ***/
94   PE_low_level_init();
95   /*** End of Processor Expert internal initialization.                    ***/
96
97   for(;;) {
98     Test();
99   }
100
```

Tutorials

Tutorials

Java Development

Create a Hello World application
Learn how to create a simple Java application that prints "Hello world!"

Create a Hello World SWT application
Learn how to create a standalone SWT Java application that displays a window to the user.

Web development
The Eclipse Web Tools Platform (WTP) project extends the Eclipse platform with tools for developing Web and Java EE applications. In these pages you will find a set of courses designed to teach anything from XML, HTML and standards based Web development to advanced topics such as Java Server Faces and Java Persistence.

Data Tools Platform

DTP Apache Derby tutorial
The Apache Derby tutorial demonstates some basic DTP features to get you up and running quickly.

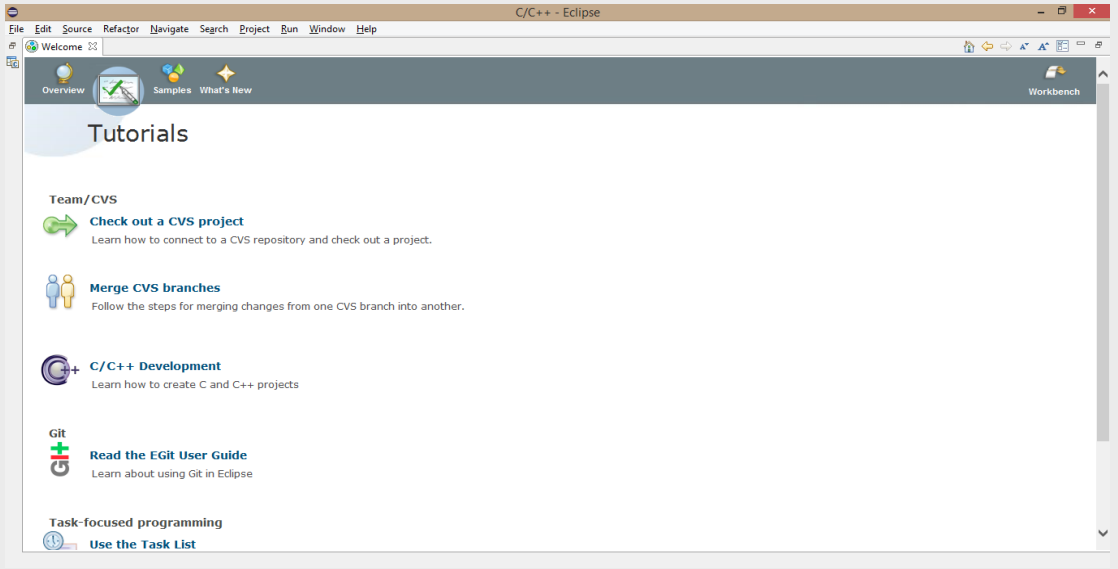
Team Support with Git

Read the EGit User Guide
Learn about using Git in Eclipse

Task-focused programming

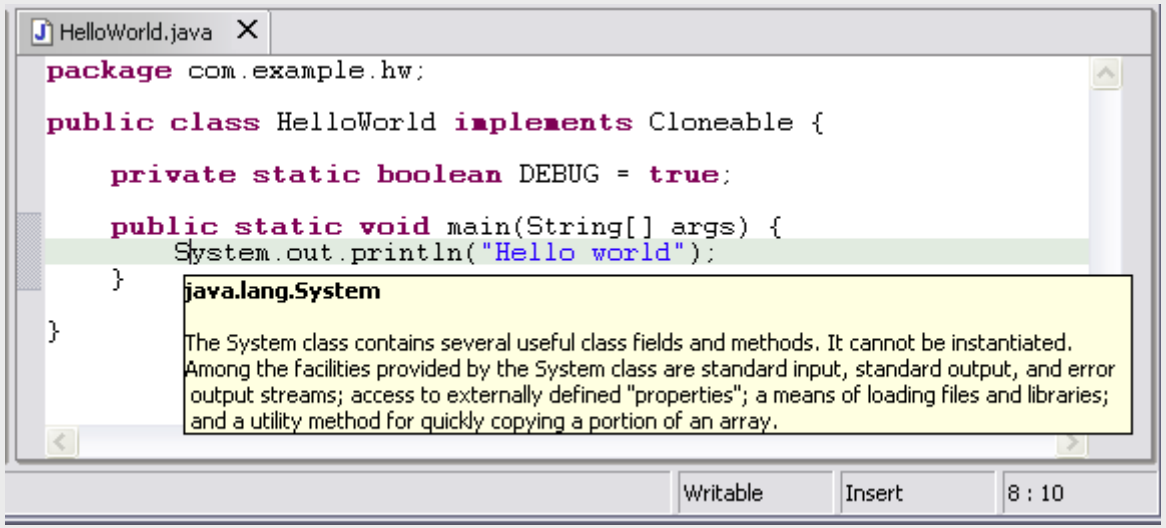
Use the Task List
Learn how to create tasks and queries

Visit the Mylyn homepage
Learn about the benefits of task-focused programming with Mylyn



Java Editor

- Hovering over identifier shows Javadoc spec

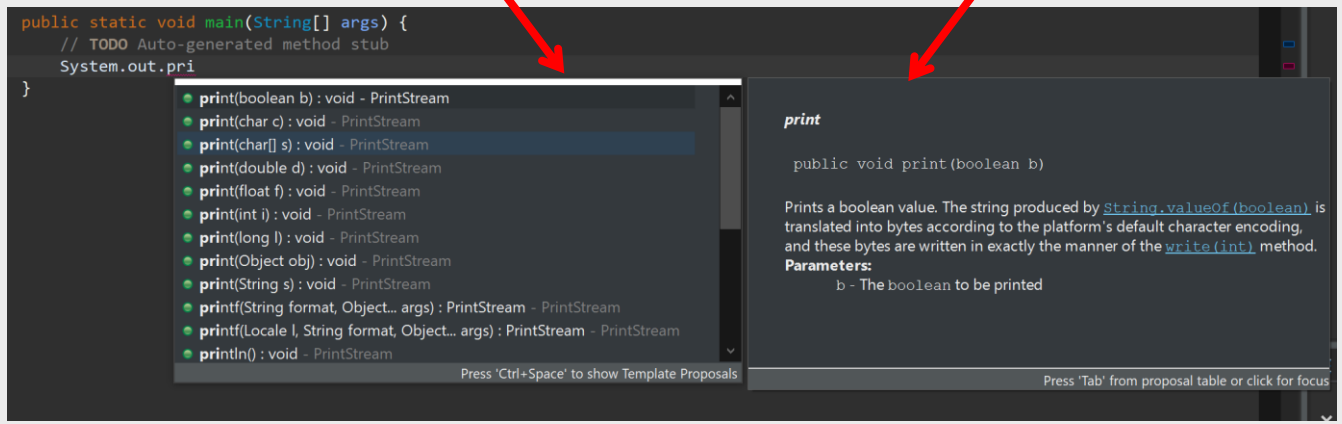


Java editor

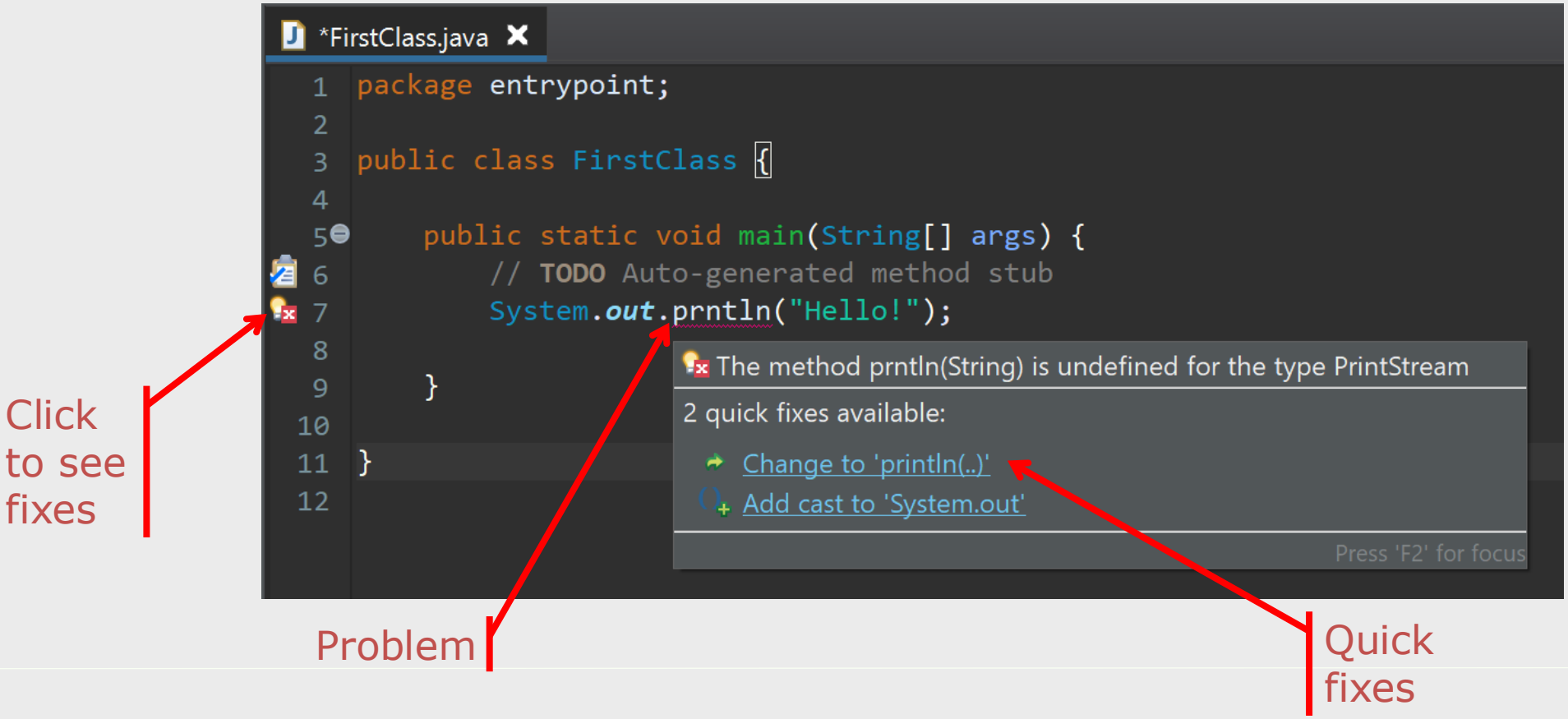
- Method completion in Java editor

List of plausible methods

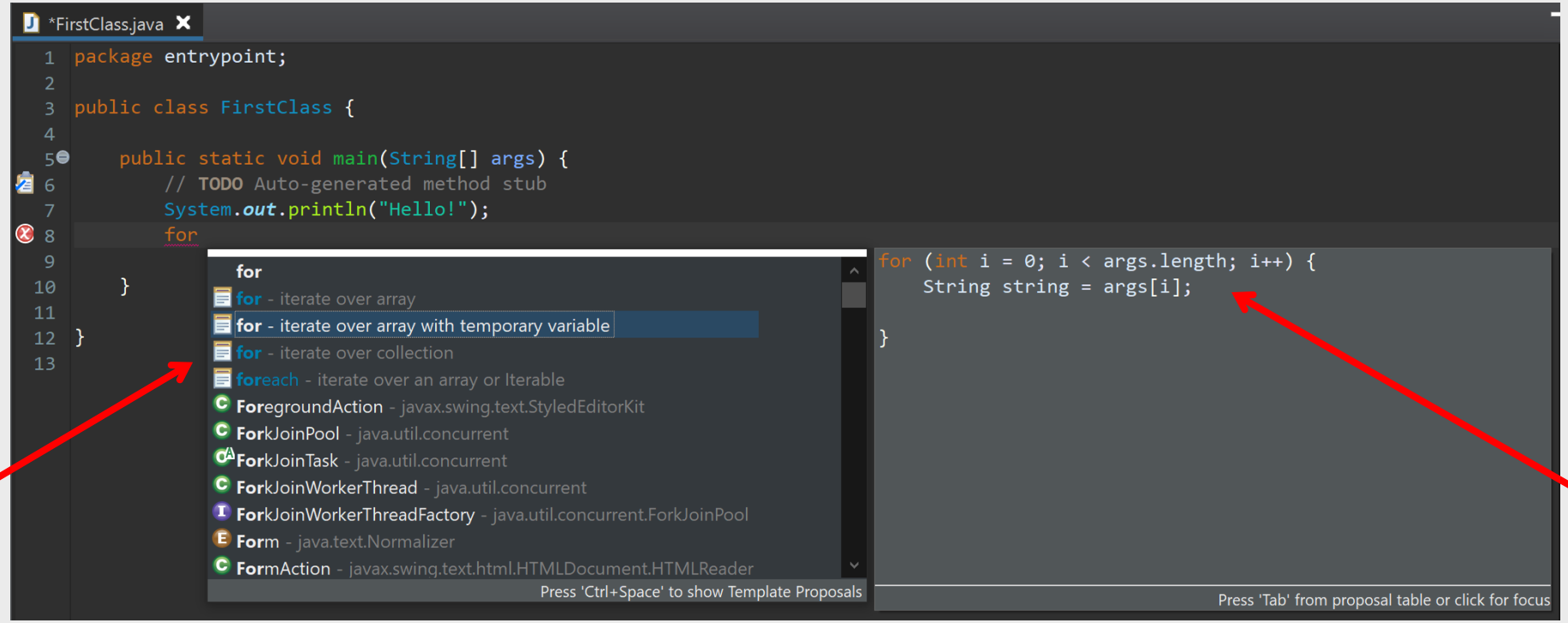
Doc for method



Java Editor


*FirstClass.java
1 package entrypoint;
2
3 public class FirstClass {
4
5 public static void main(String[] args) {
6 // TODO Auto-generated method stub
7 System.out.prntln("Hello!");
8
9 }
10
11 }
12
The method prntln(String) is undefined for the type PrintStream
2 quick fixes available:
Change to 'println(..)'
Add cast to 'System.out'
Press 'F2' for focus
Click to see fixes
Problem
Quick fixes

Code templates help



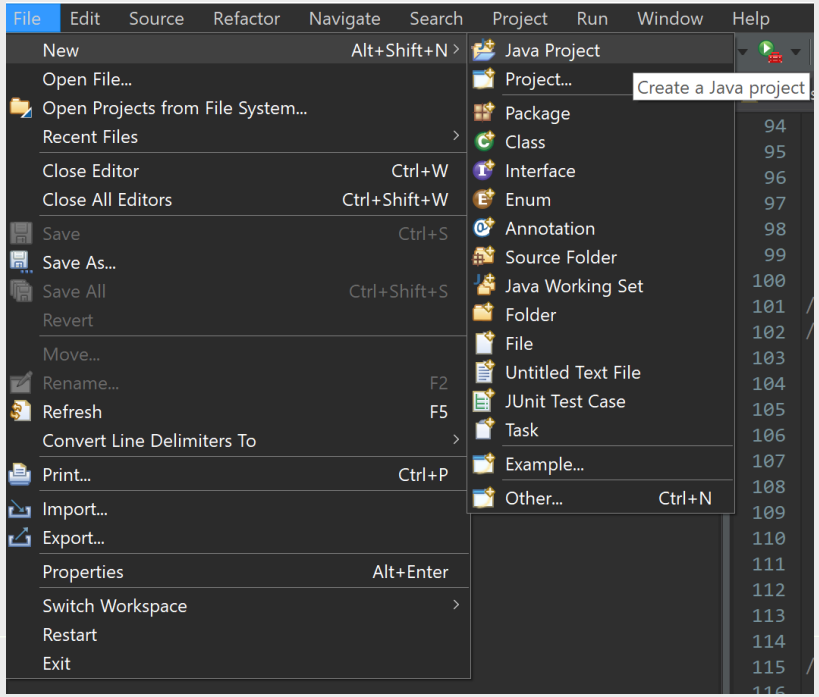
Why we really need Eclipse?

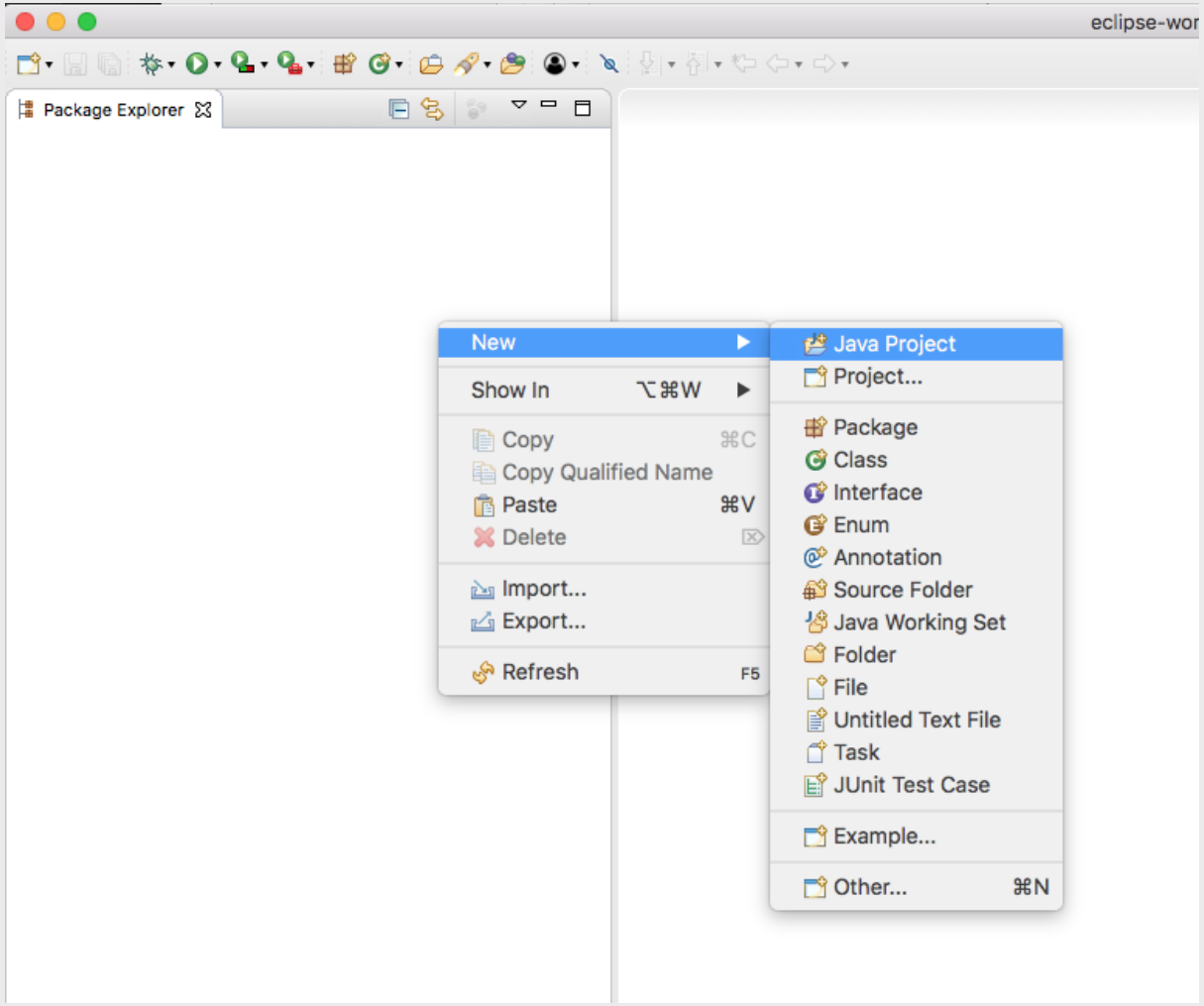
- Simplification of writing code. All attention - to the idea
- Helps to write good readable Java code (Formatting, Refactoring)
- Simplification of running, debugging process

Start coding

Creating a project

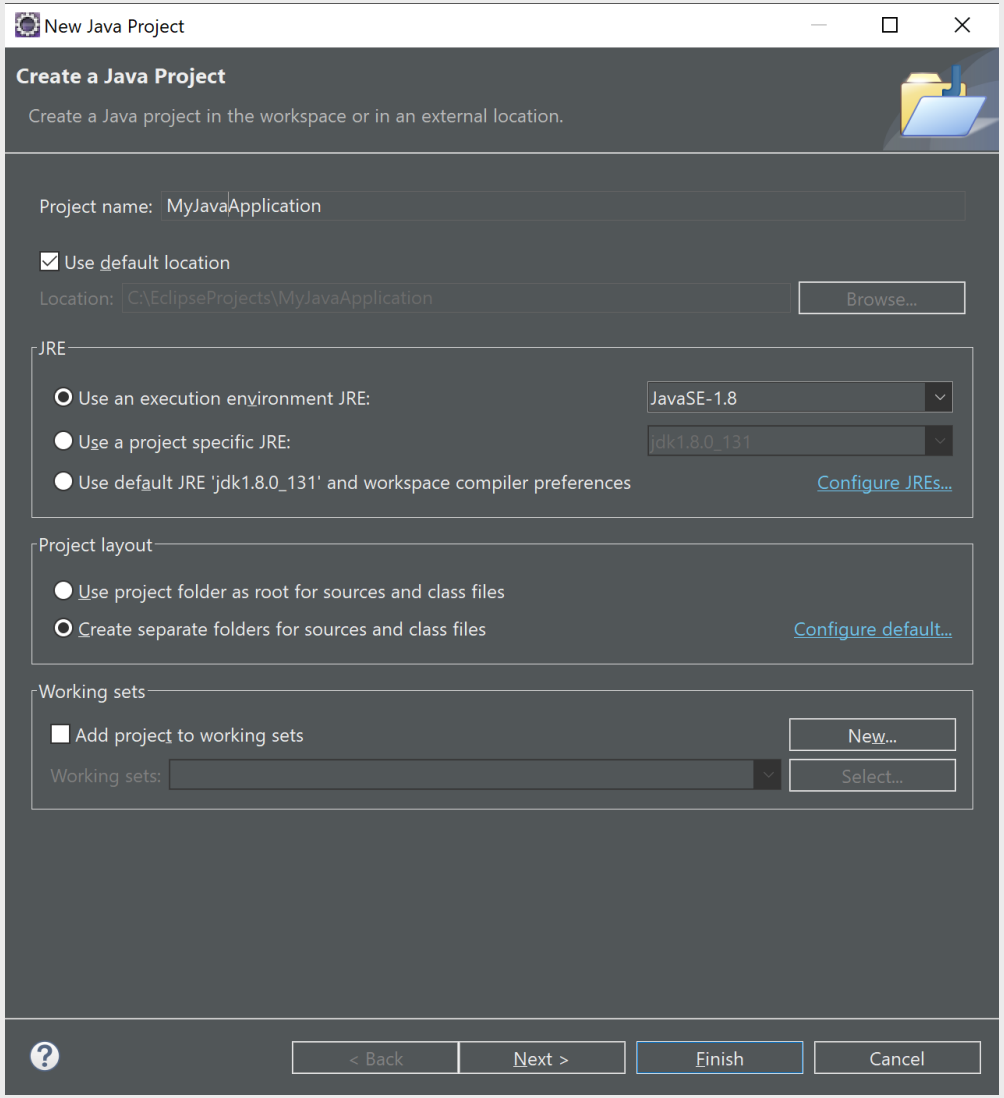
- Go to File->New->Java Project





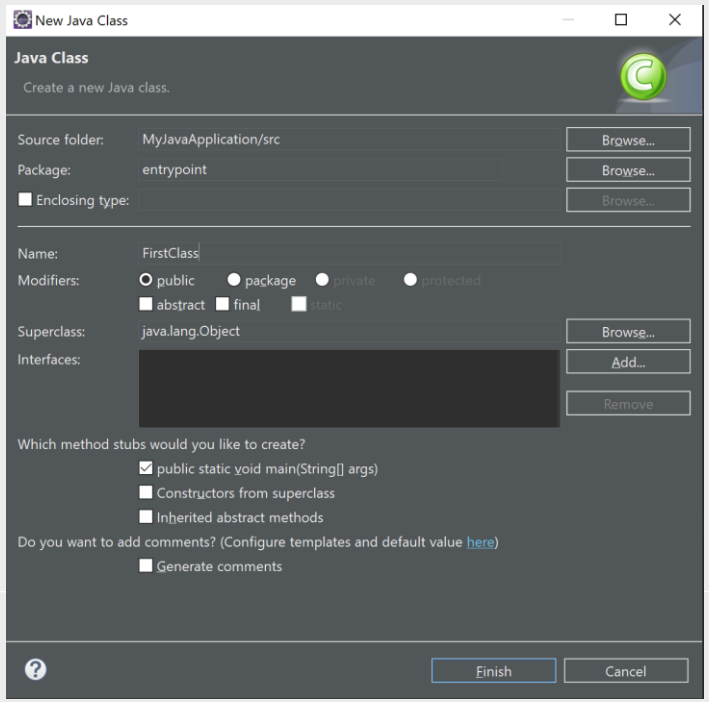
Create a project

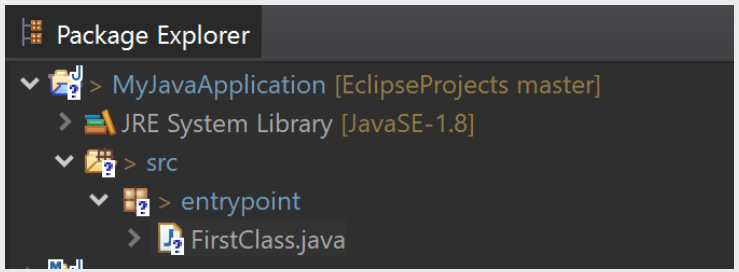
- Type a name for the Project and click Finish
  - This has nothing to do with the code, but it should be descriptive enough for you to know what code is in the project

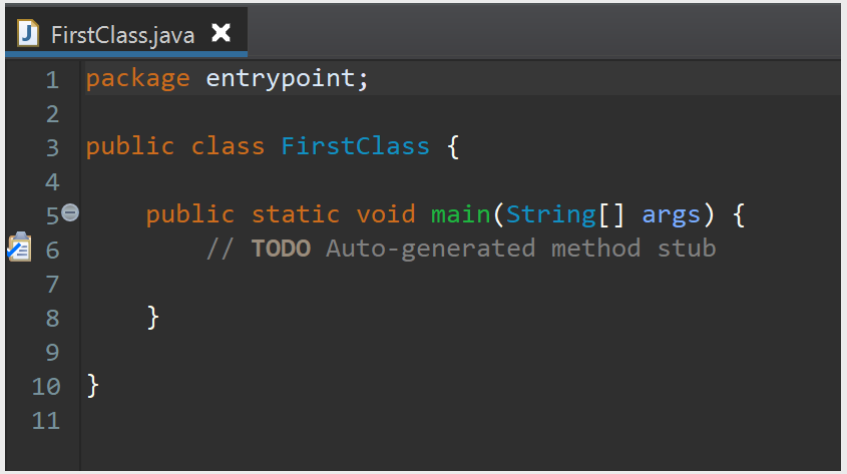


Create a Class

- Expand the Project by clicking the arrow next to the name of the project you just created
- Right-click on “src” and go to New->Class

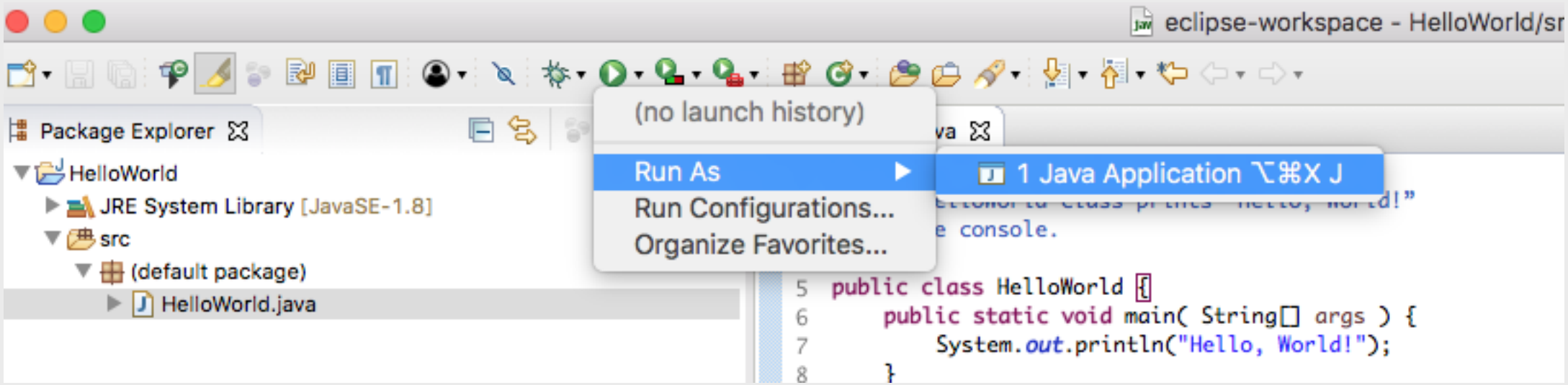






Running the code

- Click the green play button at the top to run your code
  - If your code doesn't compile, you will have error messages at the bottom of the screen
  - If your code compiles, your output will be at the bottom of the screen



Program output

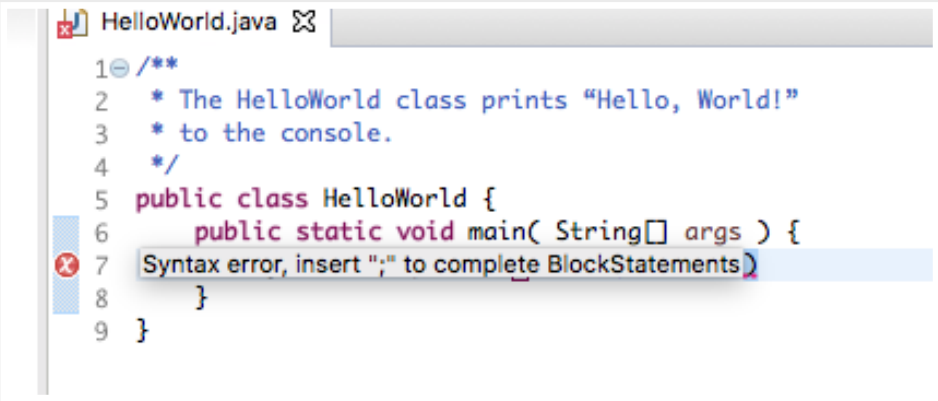
- Check your console at the bottom
  - Text output, if any, appears here
  - Compile errors will appear here as well



Compile Error!

- When you try and run without the proper syntax (code structure) the compiler will halt with an error message
  - Tells you the line where the error was detected
    - HelloWorld.java:7
- Eclipse is smart enough to detect this error before you compile, though
  - Standard text editors could not do this!



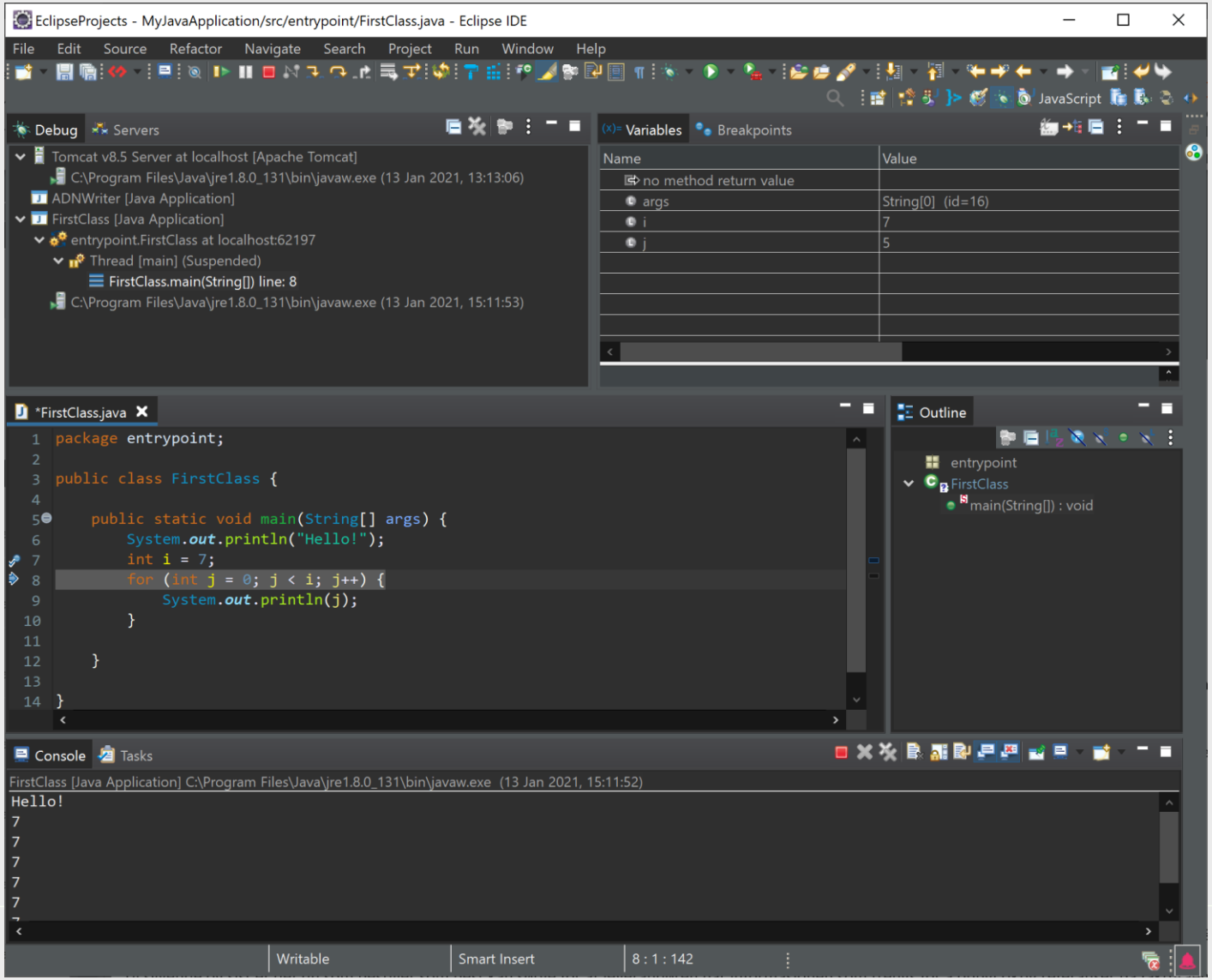


Debugging

1. Add breakpoint with right click on line number
2. Right clicking on the Java editor class file from Package explorer. Select **Debug As** → **Java Application** or use the shortcut **Alt + Shift + D, J** instead.
3. Click Step into (or press F5) or Step over (or press F6) to move on the next step in the microflow

Debugger perspective

- The debug perspective offers additional views:
  - **Debug view** – Visualizes call stack and provides operations on that.
  - **Breakpoints view** – Shows all the breakpoints.
  - **Variables/Expression view** – Shows the declared variables and their values.
  - **Display view** – Allows to Inspect the value of a variable, expression or selected text during debugging.
  - **Console view** – Program output is shown here.



Questions?